

SmartStock+

MCO 2 Final Presentation — Speaking Script & Role Assignments

Team 2 · Mobile Programming 2 · Instructor: Mr. Aaron Jude Pael · May 19, 2026

Target length: ~11–12 minutes (≈6 min slides + ≈4 min live demo + Q&A). Deck: **SmartStock+_MCO2_Presentation.pptx** (9 slides). **[SAY]** = spoken (keep it natural, don't read robotically). **[DO]** = action. **[TIME]** = running budget.

Role Assignments — Team 2 (13 members)

1	Jhon Clarence B. Rulona	Lead — Slide 1 (Title/Intro) + Q&A; lead (technical)
2	Cris Gerard O. Carpon	Speaker — Slide 2 (Project Overview)
3	Jasper Keith P. Teorica	Speaker — Slide 3 (Architecture) + Q&A; support
4	Mark Jacob S. Allero	Speaker — Slide 4 (Cloud / API Integration)
5	Leopoldo C. Villaflores III	Demo Narrator — Slide 5 (speaks the live demo)
6	John Dennis Y. Chan	Demo Driver — Slide 5 (operates the device)
7	Dustin Aaron D. Tocayon	Speaker — Slide 8 (Challenges & Learnings)
8	Angel Mae S. Dura	Speaker — Slide 9 (Conclusion & acknowledgment)
9	Heizel D. Panugaling	Deck operator — advances slides, screenshot fallback
10	Justin Jamaica P. Julaton	Supabase operator — 2nd screen / dashboard proof
11	Roselyn B. Deguino	Timekeeper — cue cards, keeps the 12-min budget
12	Lou Ariane Mae B. Delos Reyes	Backup device + connectivity (airplane-mode demo)
13	Rynel Jay V. Vallejos	Q&A; support — notes questions, assists answers

Everyone faces the audience. Non-speaking members stand with the team, ready for their support role and Q&A.

Slide 1 — Title

Opening

Jhon Clarence B. Rulona

0:00–0:45

[DO] App already open on the device, mirrored to the screen.

"Good morning, Mr. Pael. We're Team 2, and this is our MCO 2 final presentation for **SmartStock+** — an offline-first inventory and asset management system for Android. In MCO 1 we built the offline foundation. For MCO 2 we added a full cloud backend with secure multi-tenant sync. I'll start with an overview, then my teammates will walk through the architecture, the cloud integration, a live demo, and what we learned."

[DO] Advance to Slide 2.

Slide 2 — Project Overview

Introduction

Cris Gerard O. Carpon

0:45–2:00

"The problem: small businesses, school labs, and field teams still track equipment on paper or spreadsheets — and those break the moment the network is unreliable. SmartStock+ replaces that. The key design decision is **offline-first**: the app is fully usable with no connection, and syncs to the cloud automatically when it's back online. Our users are small businesses, school and IT labs, field teams, and asset custodians. The core modules: inventory CRUD with asset codes, barcode and QR scanning with per-unit checkout, usage reports with CSV and PDF export, dashboard charts and low-stock alerts, a stock-take flow, and — new in MCO 2 — multi-tenant cloud sync and authentication."

[DO] Advance to Slide 3.

Slide 3 — Architecture

How it's built

Jasper Keith P. Teorica

2:00–3:30

"Architecturally we follow **MVVM with the Repository pattern**. The Compose UI talks only to a ViewModel, which exposes state with StateFlow. The ViewModel talks to a single Repository — the only thing that touches data: Room locally, and the cloud sync source remotely. The important part is the flow: every write hits **Room instantly**, so the UI never blocks on the network. A WorkManager job — SyncWorker — reconciles with Supabase in the background, with last-write-wins, soft-deletes, and **natural-key matching** on asset code and name, which keeps sync duplicate-free even after a reinstall or on a second device. Everything is wired with Hilt, keeping data, domain, and UI cleanly separated."

[DO] Advance to Slide 4.

Slide 4 — Cloud / API Integration

Supabase backend

Mark Jacob S. Allero

3:30–5:00

"For the cloud we use **Supabase** — a REST-API backend: PostgreSQL through PostgREST, plus GoTrue auth, object Storage, and Edge Functions, consumed from Kotlin with supabase-kt over Ktor. Auth supports email/password with session persistence, password reset, and biometric unlock; an Admin provisions Staff through a secure Edge Function. Sync is bidirectional across six tables with natural-key reconciliation so we never get duplicates, and item photos sync across devices through **Supabase Storage**, scoped per team. The security highlight is **multi-tenant isolation**: every row carries a team_id, and PostgreSQL Row-Level Security — backed by a SECURITY DEFINER my_team() function — guarantees each account only ever reads its own team's data."

[DO] Point to the Supabase screenshot, then advance to Slide 5 and switch the screen to the DEVICE.

Slide 5 — UI Demo (LIVE)

Live walkthrough

Narrator: Villaflores · Driver:
Chan

5:00–9:00

Core of the presentation. Narrate every tap. If anything stalls, switch to Slides 6–7 screenshots and keep talking — never debug live.

1 · Auth & multi-tenant isolation (~45s)

[DO] Show redesigned login; tap password field to show it scrolls above the keyboard. Register a fresh account.

"Notice the inventory is **empty** — and in Profile, the team list shows **only this account**. That proves our multi-tenant isolation: a new account never sees another team's data."

2 · Create with photo + barcode (~60s)

[DO] Add an item with name/category/quantity, attach a camera photo, scan/link a barcode, save.

"That write went to the local Room database instantly — no spinner, works even offline."

3 · Cloud sync proof (~45s)

[DO] Show the sync badge / pull-to-refresh. Switch to the Supabase dashboard (Justin), refresh inventory_items.

"There's the row we just created, with its team_id — and the photo is in the Supabase Storage bucket. It synced across the network."

4 · Notification + Undo (~45s)

[DO] Edit quantity below threshold → low-stock notification. Delete an item → tap UNDO to restore.

"Low-stock alerts, and a safe undo on destructive actions."

5 · Profile depth (~45s)

[DO] Profile → toggle theme; Export Data → CSV share sheet; open Security Settings dialog.

"The Profile screen is fully functional — theme, CSV export, security posture, and team management for admins."

[DO] Switch back to the deck; advance to Slide 8. (Slides 6–7 are screenshot fallbacks — present only if the live demo failed.)

Slide 8 — Challenges & Learnings

What we learned

Dustin Aaron D. Tocayon

9:00–10:30

"A few real challenges we solved. **One — Room-to-REST sync:** the database stores time as epoch-millis, the API uses ISO-8601; we built a mapping layer and last-write-wins resolution. **Two — our biggest learning, multi-tenant isolation:** in testing a new account could see another admin's data; we root-caused it to an over-broad `is_admin()` RLS policy plus a cache not wiped on logout, and fixed it with strict per-team RLS and a cache wipe on account switch. **Three — duplicate-free sync:** client-UUID-only keying duplicated rows on reinstall; fixed by reconciling by natural key. **Four — cross-device images:** local file paths are meaningless elsewhere, so photos moved to per-team Supabase Storage. Best practices throughout: MVVM, Repository, StateFlow, Hilt, lifecycle-aware APIs, WorkManager, and RLS-backed security."

[DO] Advance to Slide 9.

Slide 9 — Conclusion & Future

Wrap-up

Angel Mae S. Dura

10:30–11:30

"To summarize: SmartStock+ is a complete offline-first inventory system — full CRUD, scanning, reporting, charts, notifications, biometric auth, and secure multi-tenant Supabase sync with cross-device image storage. It stays fully usable with zero connectivity and reconciles automatically once back online. That was our core goal, and we achieved it. Future improvements: hardware-backed biometrics with the Android Keystore, an admin web panel, real-time updates, FCM push notifications, and an audit log. Thank you, Mr. Pael, for your guidance throughout Mobile Programming 2. We'd be happy to answer any questions."

[DO] Stop. Stay on Slide 9 for Q&A.;

Q&A — Anticipated Questions · (Jhon Clarence Rulona & Jasper Teorica lead; Rynel Jay assists)

Q: Why Supabase and not Firebase? — "The requirement allows Firebase or a REST API. Supabase IS a REST-API backend (PostgREST over PostgreSQL) plus managed auth and storage. We chose it because PostgreSQL Row-Level Security gives database-enforced multi-tenant isolation — stronger than app-level checks."

Q: How does it work offline? — "Room is the single source of truth. Every read and write is local and instant. A WorkManager job syncs opportunistically when connected, so the network is never on the critical path."

Q: Two devices edit the same item? — "Last-write-wins on an updatedAt timestamp, with natural-key reconciliation so the same logical item never duplicates across devices."

Q: Is biometric login real authentication? — "It's a local unlock for an already-established session — biometrics can't authenticate against the server alone. The production-grade version, in our future improvements, binds the session token to a hardware Keystore key."

Q: How is one team's data kept private? — "Every row has a team_id. PostgreSQL RLS policies, backed by a SECURITY DEFINER my_team() function, make it physically impossible for a query to return another team's rows — enforced at the database, not the app."

Pre-Demo Checklist (before presenting)

- supabase/01_schema.sql re-run (RLS + dedup + Storage)
- Release APK rebuilt & installed; old app uninstalled first (new icon shows)
- Test account ready; one item with a photo pre-synced
- Screenshots pasted into Slides 4, 6, 7 (placeholder boxes deleted)
- Device on Wi-Fi; screen mirroring tested
- Supabase dashboard logged in on the 2nd screen (Justin)
- Airplane-mode toggle rehearsed (offline → sync)
- One full timed dry run completed with all speakers